SET TRANSACTION ISOLATION LEVEL (Transact-SQL)

Applies to: SQL Server Azure SQL Database Azure SQL Managed Instance Azure Synapse Analytics SQL database in Microsoft Fabric

Controls the locking and row versioning behavior of Transact-SQL statements issued by a connection to SQL Server.

Transact-SQL syntax conventions

Syntax

Syntax for SQL Server, Azure SQL Database, and SQL database in Microsoft Fabric.

syntaxsql

```
SET TRANSACTION ISOLATION LEVEL
    { READ UNCOMMITTED
    | READ COMMITTED
    | REPEATABLE READ
    | SNAPSHOT
    | SERIALIZABLE
    }
```

Syntax for Azure Synapse Analytics.

syntaxsql

```
SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED
```

> **Note**
>
> Azure Synapse Analytics implements ACID transactions. The default isolation level is `READ UNCOMMITTED`. You can change it to `READ COMMITTED SNAPSHOT ISOLATION` by turning `ON` the `READ_COMMITTED_SNAPSHOT` database option for a user database when connected to the `master` database. Once enabled, all transactions in this database are executed under `READ`

`COMMITTED SNAPSHOT ISOLATION` and the setting `READ UNCOMMITTED` at the session level isn't honored. For more information, see **ALTER DATABASE SET options (Transact-SQL)**.

Arguments

READ UNCOMMITTED

Specifies that statements can read rows that were modified by other transactions but not yet committed.

Transactions running at the `READ UNCOMMITTED` level don't issue shared locks to prevent other transactions from modifying data read by the current transaction. `READ UNCOMMITTED` transactions are also not blocked by exclusive locks that would prevent the current transaction from reading rows that were modified but not committed by other transactions. When this option is set, it's possible to read uncommitted modifications, which are called dirty reads. Values in the data can be changed and rows can appear or disappear in the data set before the end of the transaction. This option has the same effect as setting `NOLOCK` on all tables in all `SELECT` statements in a transaction. This is the least restrictive of the isolation levels.

In SQL Server, you can also minimize locking contention while protecting transactions from dirty reads of uncommitted data modifications using either:

- The `READ COMMITTED` isolation level with the `READ_COMMITTED_SNAPSHOT` database option set to `ON`.
- The `SNAPSHOT` isolation level. For more information about snapshot isolation, see Snapshot Isolation in SQL Server.

READ COMMITTED

Specifies that statements can't read data that was modified but not committed by other transactions. This prevents dirty reads. Data can be changed by other transactions between individual statements within the current transaction, resulting in nonrepeatable reads or phantom data. This option is the SQL Server default.

The behavior of `READ COMMITTED` depends on the setting of the `READ_COMMITTED_SNAPSHOT` database option:

- If `READ_COMMITTED_SNAPSHOT` is set to `OFF` (the default on SQL Server), the Database Engine uses shared locks to prevent other transactions from modifying rows while the current transaction is running a read operation. The shared locks also block the statement from reading rows modified by other transactions until the other transaction is completed. The shared lock type determines when it is released. Row locks are released before the next row is processed. Page locks are released when the next page is read, and table locks are released when the statement finishes.

- If `READ_COMMITTED_SNAPSHOT` is set to `ON`, the Database Engine uses row versioning to present each statement with a transactionally consistent snapshot of the data as it existed at the start of the statement. Locks aren't used to protect the data from updates by other transactions.
  - `READ_COMMITTED_SNAPSHOT ON` is the default on Azure SQL Database and SQL database in Microsoft Fabric.

> **ⓘ Important**
>
> Choosing a transaction isolation level doesn't affect the locks acquired to protect data modifications. A transaction always gets an exclusive lock on any data it modifies, and holds that lock until the transaction completes, regardless of the isolation level set for that transaction. Additionally, an update made at the `READ COMMITTED` isolation level uses update locks on the data rows selected, whereas an update made at the `SNAPSHOT` isolation level uses row versions to select rows to update. For read operations, transaction isolation levels primarily define the level of protection from the effects of modifications made by other transactions. For more information, see Transaction Locking and Row Versioning Guide.

Snapshot isolation supports FILESTREAM data. Under snapshot isolation mode, FILESTREAM data read by any statement in a transaction is the transactionally consistent version of the data that existed at the start of the transaction.

When the `READ_COMMITTED_SNAPSHOT` database option is `ON`, you can use the `READCOMMITTEDLOCK` table hint to request shared locking instead of row versioning for individual statements in transactions running at the `READ COMMITTED` isolation level.

> **ⓘ Note**

When you set the `READ_COMMITTED_SNAPSHOT` option, only the connection executing the `ALTER DATABASE` command is allowed in the database. There must be no other open connection in the database until `ALTER DATABASE` is complete. The database doesn't have to be in single-user mode.

REPEATABLE READ

Specifies that statements can't read data that was modified but not yet committed by other transactions, and that no other transactions can modify data that was read by the current transaction until the current transaction completes.

Shared locks are placed on all data read by each statement in the transaction and are held until the transaction completes. This prevents other transactions from modifying any rows that were read by the current transaction. Other transactions can insert new rows that match the search conditions of statements issued by the current transaction. If the current transaction then retries the statement, it retrieves the new rows, which results in phantom reads. Because shared locks are held to the end of a transaction instead of being released at the end of each statement, concurrency is lower than the default `READ COMMITTED` isolation level. Use this option only when necessary.

SNAPSHOT

Specifies that data read by any statement in a transaction is the transactionally consistent version of the data that existed at the start of the transaction. The transaction can only recognize data modifications that were committed before the start of the transaction. Data modifications made by other transactions after the start of the current transaction aren't visible to statements executing in the current transaction. The effect is as if the statements in a transaction get a snapshot of the committed data as it existed at the start of the transaction.

Except when a database is being recovered, `SNAPSHOT` transactions don't request locks when reading data. `SNAPSHOT` transactions reading data don't block other transactions from writing data. Transactions writing data don't block `SNAPSHOT` transactions from reading data.

During the roll-back phase of a database recovery, `SNAPSHOT` transactions request a lock if an attempt is made to read data that is locked by another transaction that is being rolled back. The

`SNAPSHOT` transaction is blocked until that transaction is rolled back. The lock is released immediately after it is granted.

The `ALLOW_SNAPSHOT_ISOLATION` database option must be set to `ON` before you can start a transaction that uses the `SNAPSHOT` isolation level. If a transaction using the `SNAPSHOT` isolation level accesses data in multiple databases, `ALLOW_SNAPSHOT_ISOLATION` must be set to `ON` in each database.

A transaction can't be set to `SNAPSHOT` isolation level that started with another isolation level; doing so causes the transaction to abort. If a transaction starts in the `SNAPSHOT` isolation level, you can change it to another isolation level and then back to `SNAPSHOT`. A transaction starts the first time it accesses data.

A transaction running under `SNAPSHOT` isolation level can view changes made by that transaction. For example, if the transaction performs an `UPDATE` on a table and then issues a `SELECT` statement against the same table, the modified data is included in the result set.

> **Note**
>
> Under snapshot isolation mode, FILESTREAM data read by any statement in a transaction is the transactionally consistent version of the data that existed at the start of the transaction, not at the start of the statement.

SERIALIZABLE

Specifies the following conditions:

- Statements can't read data that was modified but not yet committed by other transactions.
- No other transactions can modify data that was read by the current transaction until the current transaction completes.
- Other transactions can't insert new rows with key values that would fall in the range of keys read by any statements in the current transaction until the current transaction completes.

Range locks are placed in the range of key values that match the search conditions of each statement executed in a transaction. This blocks other transactions from updating or inserting any rows that would qualify for any of the statements executed by the current transaction. This

means that if any of the statements in a transaction are executed a second time, they read the same set of rows. The range locks are held until the transaction completes. This is the most restrictive of the isolation levels because it locks entire ranges of keys and holds the locks until the transaction completes. Because concurrency is lower, use this option only when necessary. This option has the same effect as setting `HOLDLOCK` on all tables in all `SELECT` statements in a transaction.

Remarks

Only one of the isolation level options can be set at a time, and it remains set for that connection until it's explicitly changed. All read operations performed within the transaction operate under the rules for the specified isolation level unless a table hint in the `FROM` clause of a statement specifies different locking or versioning behavior for a table.

The transaction isolation levels define the type of locks acquired on read operations. Shared locks acquired for `READ COMMITTED` or `REPEATABLE READ` are generally row locks, although the row locks can be escalated to page or table locks if a significant number of the rows in a page or table are referenced by the read. If the transaction modifies a row after it was read, the transaction acquires an exclusive lock to protect that row, and the exclusive lock is retained until the transaction completes. For example, if a `REPEATABLE READ` transaction has a shared lock on a row, and the transaction then modifies the row, the shared row lock is converted to an exclusive row lock.

With one exception, you can switch from one isolation level to another at any time during a transaction. The exception occurs when changing from any isolation level to `SNAPSHOT` isolation. Doing this causes the transaction to fail and roll back. However, you can change a transaction started in `SNAPSHOT` isolation to any other isolation level.

When you change a transaction from one isolation level to another, resources that are read after the change are protected according to the rules of the new level. Resources that are read before the change continue to be protected according to the rules of the previous level. For example, if a transaction changed from `READ COMMITTED` to `SERIALIZABLE`, the shared locks acquired after the change are now held until the end of the transaction.

If you issue `SET TRANSACTION ISOLATION LEVEL` in a stored procedure or trigger, when the object returns control the isolation level is reset to the level in effect when the object was invoked. For example, if you set `REPEATABLE READ` in a batch, and the batch then calls a stored procedure that

sets the isolation level to `SERIALIZABLE`, the isolation level setting reverts to `REPEATABLE READ` when the stored procedure returns control to the batch.

> **Note**
>
> User-defined functions and common language runtime (CLR) user-defined types can't execute `SET TRANSACTION ISOLATION LEVEL`. However, you can override the isolation level by using a table hint. For more information, see Table Hints (Transact-SQL).

When you use `sp_bindsession` to bind two sessions, each session retains its isolation level setting. Using `SET TRANSACTION ISOLATION LEVEL` to change the isolation level setting of one session doesn't affect the setting of any other sessions bound to it.

`SET TRANSACTION ISOLATION LEVEL` takes effect at execute or run time, and not at parse time.

Optimized bulk load operations on heaps block queries that are running under the following isolation levels:

- `SNAPSHOT`
- `READ UNCOMMITTED`
- `READ COMMITTED` using row versioning

Conversely, queries that run under these isolation levels block optimized bulk load operations on heaps. For more information about bulk load operations, see Bulk Import and Export of Data (SQL Server).

FILESTREAM-enabled databases support the following transaction isolation levels.

Expand table

Isolation level	Transact-SQL access	File system access
Read uncommitted	SQL Server	Unsupported
Read committed	SQL Server	SQL Server
Repeatable read	SQL Server	Unsupported
Serializable	SQL Server	Unsupported

Isolation level	Transact-SQL access	File system access
Read committed snapshot	SQL Server	SQL Server
Snapshot	SQL Server	SQL Server

Examples

The following example sets the `TRANSACTION ISOLATION LEVEL` for the session. For each Transact-SQL statement that follows, SQL Server holds all of the shared locks until the end of the transaction.

SQL

```
USE AdventureWorks2022;
GO

SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
GO

BEGIN TRANSACTION;
GO

SELECT *
FROM HumanResources.EmployeePayHistory;
GO

SELECT *
FROM HumanResources.Department;
GO

COMMIT TRANSACTION;
GO
```

Related content

- ALTER DATABASE (Transact-SQL)
- DBCC USEROPTIONS (Transact-SQL)
- SELECT (Transact-SQL)
- SET Statements (Transact-SQL)
- Table hints (Transact-SQL)

Last updated on 22/09/2026